CSC3042F 2026
Assignment 2

Encoder-Decoder Models for Grapheme-to-Phoneme Conversion

Total marks: 30

A *grapheme* is the smallest unit of written language (letters in the alphabet), while *phoneme* are the smallest units of sound in a language. Grapheme-to-phoneme (G2P) conversion is the task of mapping a written word (a sequence of characters – graphemes) to its pronunciation (a sequence of speech sounds – phonemes). For example:

`h e l l o` ⟶ `HH EH1 L OW0` `e n o u g h` ⟶ `IH0 N AH1 F`

The phonemes above are written in ARPAbet notation, a standard phoneme alphabet for English. Each symbol denotes a single distinctive sound: `B` and `P` are separate phonemes because swapping them changes *bat* (`B AE1 T`) into *pat* (`P AE1 T`). ARPAbet has 39 base phonemes covering the full range of English consonants and vowels. Vowels carry a numeric *stress marker*: 0 = unstressed, 1 = primary stress, 2 = secondary stress. This gives roughly 84 distinct phoneme tokens in total.

G2P conversion is a core component of text-to-speech systems. Before a system like Siri, Google Assistant, or a screen reader can synthesise speech, it must know how to pronounce every word. For common words a lookup table suffices, but for rare words, names, and technical terms, a model is needed. G2P is also used in automatic speech recognition (speech-to-text) to build pronunciation lexicons. English spelling is highly irregular. The letter sequence `ough` is pronounced at least five different ways: *through* (`TH R UW0`), *dough* (`D OW1`), *cough* (`K AO1 F`), *rough* (`R AH1 F`), and *thought* (`TH AO1 T`). A model cannot simply apply letter-to-sound rules – it must learn statistical speech sound patterns from data.

G2P can be framed as a sequence-to-sequence task. The input is a sequence of characters and the output is a sequence of ARPAbet symbols. As the examples above show, the input and output sequences often have different lengths. G2P has several properties that make it ideal to model with encoder-decoder architectures:

- The source alphabet has 26 characters and the target alphabet has ∼84 phoneme tokens. Both are small and fixed.
- Source and target sequences are short. Most English words are 3–10 characters and 1–12 phonemes long, so training is relatively fast, even on a laptop CPU.
- Evaluation is simple. Unlike machine translation, where many translations can be valid, each word has a single target pronunciation, so predictions can be compared directly.

In class, we discussed the hidden bottleneck problem of LSTM encoder-decoders and how you can overcome it by passing a context vector to every decoder step. Your task in this assignment is to implement an LSTM encoder-decoder with three options: (1) no context vector, (2) using the final encoder hidden state as context vector at every decoder step, and (3) computing the context vector dynamically with cross-attention at every decoder step. Using G2P conversion as a task, you will compare these three variants in terms of performance. In the process, you will observe the hidden-state bottleneck firsthand and see how attention overcomes it.

You may use standard libraries like `PyTorch`, `numpy`, and `matplotlib`. You **may not** use `nn.LSTM`, `nn.LSTMCell`, or any other built-in PyTorch classes for RNNs. You may only use PyTorch classes for individual layers, such as `nn.Linear` and `nn.Embedding`, and any PyTorch optimisers and data utilities. You may also adapt code from the LSTM tutorial notebook.

Dataset

The dataset for this assignment is the **CMU Pronouncing Dictionary (CMUdict)**, a freely available lexicon of ∼134,000 North American English words with their ARPAbet transcriptions. The dataset is derived from the CMU Pronouncing Dictionary.

We provide train/validation/test data files on Amathuba (Resources > Datasets > A2-dataset). Download and place these three CSV files in your working directory:

File	Words	Description
`g2p_train.csv`	92,426	Training set
`g2p_val.csv`	11,553	Validation set
`g2p_test.csv`	11,554	Test set

Each file has two columns: `word` (lowercase string) and `phonemes` (space-separated ARPAbet sequence e.g. `P EY1 N T`). **You must use these exact splits.** Do not re-split the data.

Tasks

1. Data processing **(3 marks)**

Load the provided CSV files and prepare them for training.

- Build a character vocabulary from the training words only. Include four special tokens at fixed indices: `<PAD>` = 0, `<SOS>` = 1, `<EOS>` = 2, `<UNK>` = 3. Do not include validation or test data when building the vocabulary.
- Build a phoneme vocabulary from the training phoneme sequences only, using the same four special tokens at the same indices.
- Implement a PyTorch Dataset class that encodes each (word, phoneme) pair as integer index sequences. Add `<SOS>` at the start and `<EOS>` at the end of each target sequence.
- Implement `collate_fn` and create `DataLoader`s with padding: in each batch, pad all source sequences to the same length and all target sequences to the same length, with `<PAD>` = 0.
- In your notebook: print vocabulary sizes and dataset sizes; plot histograms of word length and phoneme sequence length.

2. LSTM cell

Implement the LSTM cell from scratch. You may reuse your lab implementation directly. The six gating equations and the full mathematical specification are given in Appendix A. Implement two classes with the following signatures:

```
class LSTMCell(nn.Module):
    def __init__(self, input_size: int, hidden_size: int): ...
    def forward(self, x, h, c): ...
```

3. Encoder-decoder implementation **(9 marks)**

Implement one `Encoder` class, one `Decoder` class (parameterised by context mode), and a `Seq2Seq` wrapper. The full mathematical specification for all three setups is given in Appendix A. Implement the equations exactly as stated there.

For generation, you only need to implement **greedy decoding** as discussed in class – at each step, take the argmax of the output logits as the next predicted token. You do not need to implement beam search, sampling, or any other decoding strategy.

Required classes and signatures:

```
class Encoder(nn.Module):
    def __init__(self, src_vocab_size: int, embed_dim: int,
                 hidden_size: int, num_layers: int = 1): ...

    def forward(self, src):
        # src: (batch, src_len)
        # Returns (all_h, (h_n, c_n))
        # all_h: (src_len, batch, hidden_size)
        # h_n, c_n: (num_layers, batch, hidden_size)


class Decoder(nn.Module):
    def __init__(self, tgt_vocab_size: int, embed_dim: int,
                 hidden_size: int, num_layers: int = 1,
                 context_mode: str = "none"): ...
    # context_mode in {"none", "fixed", "attn"}

    def forward(self, tgt, hidden, encoder_outputs=None):
        # tgt: (batch, tgt_len)
        # hidden: (h_0, c_0)
        #   h_0, c_0: (num_layers, batch, hidden_size)
        # encoder_outputs: optional (only needed for attention)
        #   (src_len, batch, hidden_size)
        #
        # Returns (logits, (h_n, c_n))
        # logits: (batch, tgt_len, tgt_vocab_size)
        # h_n, c_n: (num_layers, batch, hidden_size)


class Seq2Seq(nn.Module):
    def __init__(self, encoder: Encoder, decoder: Decoder): ...

    def forward(self, src, tgt, teacher_forcing: bool = True):
        # Returns logits: (batch, tgt_len-1, tgt_vocab_size)

    def greedy_decode(self, src, max_len: int = 30):
        # Returns predicted token-id sequences (one per batch element)
```

The three setups differ only in how the `Decoder` uses encoder information. They share the same `Encoder` and the same `Seq2Seq` wrapper; only `context_mode` changes. **(4 marks)**

Setup 1: No context vector (bottleneck baseline). (Included in the 4 marks above.)

The encoder's final hidden state $(\boldsymbol{h}_n^{\text{enc}}, \boldsymbol{c}_n^{\text{enc}})$ initialises the decoder, but no encoder information is provided at subsequent decode steps. All source information must flow through this single initial state, which is an information bottleneck. See Appendix A.2 for equations.

Setup 2: Fixed context vector at every decoder step. (2 marks)

The final encoder hidden state $\boldsymbol{z} = \boldsymbol{h}_n^{\text{enc}}$ is provided to the decoder cell at every decode step (not just at initialisation), entering each LSTM gate through its own learned weight matrix. This

gives the decoder a direct view of the encoded source throughout generation. See Appendix A.3 for equations.

Setup 3: Attention-based dynamic context vector. **(3 marks)**

Instead of a single fixed context vector, the decoder computes a different context vector $\boldsymbol{z}_t$ at every step by attending over all encoder hidden states. You are required to implement **dot-product attention** exactly as specified in Appendix A.4. Save the attention weight matrices $\{\boldsymbol{\alpha}_t\}$ during greedy decoding so you can visualise them in Section 8.

4. Training **(2 marks)**

Train each model with cross-entropy loss and the Adam optimiser.

- Use `nn.CrossEntropyLoss` with `ignore_index` set to the `<PAD>` index. Compute the loss over the target sequence excluding the leading `<SOS>`.
- Apply gradient clipping (`torch.nn.utils.clip_grad_norm_`, `max_norm=1.0`) at every step to prevent gradient explosions in the from-scratch LSTM.
- Use and motivate a stopping criterion in your report (fixed epochs or early stopping on validation loss). Include a plot of training and validation loss curves.

5. Hyperparameter tuning **(3 marks)**

Using **Setup 1** as the model, tune the following hyperparameters on the *validation set*. Test at least three values for each:

- Learning rate (suggested range: `1e-5` to `1e-3`).
- Embedding dimension (suggested values: 32, 64, 128).
- Hidden size (suggested values: 64, 128, 256).

You do not have to implement or experiment with depth – a 1-layer LSTM should be sufficient for this task, Report your results in a table (e.g. validation PER per configuration) and justify your chosen configuration. Use this same configuration for all three setups in Sections 6 and 7.

A note on compute budget and time constraints. It is strongly recommended that you use Colab GPUs to speed up your experiments. Some of your models will take long to train. You can either leave them running on your laptop, or use a free Colab GPU if available. To save time for hyperparameter tuning, you do not need to train each tuning configuration to convergence. A practical approach is to run each configuration for a small, fixed number of epochs (e.g. 5–10), pick the best configuration based on validation PER, and then train your three final models (Section 6) for longer (e.g. 30 epochs, or until validation PER stops improving for several epochs). If your implementation is correct, you should see accuracy increasing after a few epochs and a single short run should not take too long on a laptop CPU. Longer final training runs will take more time – this is unavoidable with deep learning.

6. Architecture comparison **(3 marks)**

Using the best hyperparameters from Section 5, train all three setups under identical conditions: same hyperparameters, same random seed, same stopping condition, same batch size.

- Plot training and validation loss curves for all three setups on shared axes.
- After training, evaluate on the test set (see Section 7) and report results in a table.
- In your report, compare and discuss:
  - Which setup converges faster? Which reaches the lowest test loss?

– Does the fixed context vector (Setup 2) improve over the bottleneck (Setup 1)? By how much?

– Does attention (Setup 3) improve further? What does this suggest?

– Analyse a few errors to better understand the performance differences between your variants. For example, find two or three test words where Setup 1 fails but Setup 3 succeeds, and comment on what they have in common.

7. Evaluation **(2 marks)**

Report the following metrics on the test set for each setup.

- **Phoneme error rate (PER).** PER measures how far a predicted phoneme sequence is from the reference, in units of phonemes. It is based on the edit distance (also known as Levenshtein distance): the minimum number of single-phoneme insertions, deletions, and substitutions needed to turn the prediction into the reference. PER for a single word is

$$\text{PER} = \frac{\text{edit distance}}{\text{length of reference}},$$

  and the reported PER is this quantity averaged over the test set. For example, if the reference is `HH EH1 L OW0` (4 phonemes) and the prediction is `HH AH1 L OW0`, one substitution is needed, giving a PER of $1/4 = 0.25$. Lower values mean better performance.

- **Word accuracy.** The fraction of test words for which the entire predicted phoneme sequence exactly matches the reference. This is a stricter metric than PER, since a single wrong phoneme makes the whole word count as incorrect.

You do not need to implement edit distance yourself. Any standard library will do. For example, `nltk.metrics.distance.edit_distance` or the `editdistance` package (`pip install editdistance`), both of which take two sequences and return their Levenshtein distance. Make sure you pass in lists of phoneme tokens, not strings, so that each phoneme (e.g. `EH1`) is treated as a single symbol rather than as separate characters.

Present both metrics in a clearly labelled table comparing the three setups.

8. Bottleneck OR attention analysis (choose ONE of the following) **(2 marks)**

(a) Length analysis. Bucket the test words by word length (e.g. 1–4, 5–7, 8–10, 11+ characters). For each bucket and each setup, compute the word accuracy. Plot these as a grouped bar chart. Discuss whether the bottleneck (Setup 1) degrade more than Setup 2 or Setup 3 as words get longer? Why or why not?

(b) Attention visualisation. For Setup 3, choose five test words (at least two of which are ≥ 8 characters long) and visualise the attention weight matrix $\boldsymbol{\alpha}$ for each. Plot a heatmap with the input character sequence on the x-axis, the output phoneme sequence on the y-axis, and $\alpha_{t,i}$ as the cell colour. Discuss what the attention pattern reveals: does the model attend to the characters that are most relevant to each output phoneme?

9. Code quality and reproducibility **(3 marks)**

- Use object-oriented design throughout: `Encoder`, `Decoder`, and `Seq2Seq` are classes; training and evaluation are functions with clear signatures. Code is readable with sensible variable names.

- Fix all random seeds (`torch.manual_seed`, `random.seed`, and `numpy.random.seed`) at the top of the notebook with the same value, so that your results can be exactly reproduced by running the notebook top-to-bottom from scratch.

- Include a `README` file with clear setup instructions (including which Python libraries are

required) and a brief description of each submitted file.

- The notebook must run end-to-end without errors on a clean Colab environment.

10. Final report **(3 marks)**

Submit a PDF report of at most **5 pages**. The report should:

- Describe your OOP design and any non-obvious implementation choices.
- Present your hyperparameter tuning results and justify your chosen configuration.
- Report and discuss all your analysis – the architecture comparison results, the bottleneck/attention analysis, and any other insightful findings.
- Include the key figures: loss curves, length-bucketed accuracy plot, and at least two attention heatmaps.

Submission requirements

Submit a single compressed archive named as your student number, e.g. `ABCDEF123.tar.xz`. The archive must contain:

- Your Jupyter notebook (`.ipynb`), which must run end-to-end.
- A `README` file with setup instructions and a description of each file.
- Your PDF report (at most 5 pages).
- Do not submit the dataset files. The tutor marking your assignment will place the data files on Amathuba (`g2p_train.csv`, `g2p_valid.csv`, `g2p_test.csv`) in a directory called `data/`, located in the same directory as your notebook. Your notebook should load the data from this relative path.

Ensure your tarball is not corrupt before submission (extract it and verify it runs). **Corrupt or non-working tarballs will not be marked and you will get zero.** A 10 % penalty is incurred for submissions up to 24 hours late. Submissions later than 24 hours receive 0 %.

Important! Academic integrity and AI usage policy

All code and analysis must be your own work. You may discuss the assignment with other students, but code sharing is prohibited.

AI usage statement (mandatory). You must include a brief AI Usage Statement at the top of your notebook and at the start of your report. A **10 % mark penalty** applies if no statement is included. The statement should declare which AI tools (if any) you used and how. A statement like "I did not use any AI tools" is acceptable if true.

For coding, you may use GenAI tools (ChatGPT, Claude, Copilot, etc.) for the following purposes:

- Debugging an error in code you wrote yourself.
- Understanding a documentation page or an online code example you found.
- Making small, targeted changes (a line or two) to your own code.

The following is not permitted:

- Asking an AI to generate any section of your implementation from scratch.
- Submitting code you do not understand.

For the report, you may use GenAI tools for the following purposes:

- Receiving critical feedback on weaknesses in your argument, structure, or clarity.
- Receiving suggestions for how to improve a passage you have already written.
- Help with planning the overall structure and organisation of your report.

The following is not permitted:

- Generating any part of the report from scratch with an AI tool.
- Copy-pasting AI-generated text directly into your report, even as a starting point. This constitutes plagiarism.
- Including fabricated claims, statistics, or references produced by an AI tool. This is academic dishonesty. Students bear full responsibility for verifying the validity and accuracy of any AI output, and "the AI said so" will not be accepted as an excuse.

NOTE: If we suspect that a submission contains AI-generated code or writing, you may be called in to explain sections of your implementation or report to the lecturer in person. Being unable to do so, or other obvious signs of improper AI use, will be treated as a violation of the academic integrity policy and may lead to further disciplinary action.

Marking rubric

Category	**Marks**
1. Data processing	3
2. LSTM cell (available in the tut, so no marks for this)	
3. Encoder-decoder implementation (total)	9
3a. Encoder & Setup 1 (bottleneck decoder)	4
3b. Setup 2 (fixed context vector)	2
3c. Setup 3 (dot-product attention)	3
4. Training	2
5. Hyperparameter tuning	3
6. Architecture comparison	3
7. Evaluation	2
8. Bottleneck or attention analysis	2
9. Code quality and reproducibility	3
10. Final report	3
Total	**30**

Appendix A — Mathematical specification

This appendix gives the complete mathematical specification of the LSTM cell and the three encoder-decoder setups. Implement the equations exactly as written. In all equations: σ denotes the sigmoid function, $\odot$ denotes element-wise multiplication, and all weight matrices (W, U, V) and bias vectors ($\boldsymbol{b}$) are learned parameters.

A.1 LSTM cell

For a single LSTM layer at time step t, given input $\boldsymbol{x}_t$, previous hidden state $\boldsymbol{h}_{t-1}$, and previous cell state $\boldsymbol{c}_{t-1}$:

$$\boldsymbol{f}_t = \sigma(W_f\,\boldsymbol{x}_t + U_f\,\boldsymbol{h}_{t-1} + \boldsymbol{b}_f) \tag{A.1}$$
$$\boldsymbol{i}_t = \sigma(W_i\,\boldsymbol{x}_t + U_i\,\boldsymbol{h}_{t-1} + \boldsymbol{b}_i) \tag{A.2}$$
$$\boldsymbol{o}_t = \sigma(W_o\,\boldsymbol{x}_t + U_o\,\boldsymbol{h}_{t-1} + \boldsymbol{b}_o) \tag{A.3}$$
$$\tilde{\boldsymbol{c}}_t = \tanh(W_c\,\boldsymbol{x}_t + U_c\,\boldsymbol{h}_{t-1} + \boldsymbol{b}_c) \tag{A.4}$$
$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t \tag{A.5}$$
$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t) \tag{A.6}$$

A.2 Setup 1: No context vector (bottleneck baseline)

Let n denote the source sequence length. The encoder runs over all source positions:

$$\boldsymbol{h}_t^{\text{enc}},\ \boldsymbol{c}_t^{\text{enc}} \;=\; f_{\text{enc}}\big(\boldsymbol{x}_t,\ \boldsymbol{h}_{t-1}^{\text{enc}},\ \boldsymbol{c}_{t-1}^{\text{enc}}\big)\,, \quad t = 1, \ldots, n. \tag{A.7}$$

The decoder is initialised from the encoder's final state and then runs *without* any further encoder input:

$$\boldsymbol{h}_0^{\text{dec}} = \boldsymbol{h}_n^{\text{enc}}, \qquad \boldsymbol{c}_0^{\text{dec}} = \boldsymbol{c}_n^{\text{enc}} \tag{A.8}$$
$$\boldsymbol{h}_t^{\text{dec}},\ \boldsymbol{c}_t^{\text{dec}} = f_{\text{dec}}\Big(\boldsymbol{y}_{t-1},\ \boldsymbol{h}_{t-1}^{\text{dec}},\ \boldsymbol{c}_{t-1}^{\text{dec}}\Big) \tag{A.9}$$

where $\boldsymbol{y}_{t-1}$ is the embedding of the previous target token. All source information must flow through the initial hidden state alone.

A.3 Setup 2: Fixed context vector at every decoder step

The encoder is unchanged (Eq. A.7). The final encoder hidden state is used as a fixed context vector:

$$\boldsymbol{z} \;=\; \boldsymbol{h}_n^{\text{enc}}. \tag{A.10}$$

The decoder is initialised as in Setup 1 (Eq. A.8). At every decode step, $\boldsymbol{z}$ enters each gate through its own learned weight matrix V. The full decoder cell equations are:

$$\boldsymbol{f}_t^{\text{dec}} = \sigma\Big(W_f^{\text{dec}}\,\boldsymbol{y}_{t-1} + U_f^{\text{dec}}\,\boldsymbol{h}_{t-1}^{\text{dec}} + V_f\,\boldsymbol{z} + \boldsymbol{b}_f^{\text{dec}}\Big) \tag{A.11}$$
$$\boldsymbol{i}_t^{\text{dec}} = \sigma\Big(W_i^{\text{dec}}\,\boldsymbol{y}_{t-1} + U_i^{\text{dec}}\,\boldsymbol{h}_{t-1}^{\text{dec}} + V_i\,\boldsymbol{z} + \boldsymbol{b}_i^{\text{dec}}\Big) \tag{A.12}$$
$$\boldsymbol{o}_t^{\text{dec}} = \sigma\Big(W_o^{\text{dec}}\,\boldsymbol{y}_{t-1} + U_o^{\text{dec}}\,\boldsymbol{h}_{t-1}^{\text{dec}} + V_o\,\boldsymbol{z} + \boldsymbol{b}_o^{\text{dec}}\Big) \tag{A.13}$$
$$\tilde{\boldsymbol{c}}_t^{\text{dec}} = \tanh\Big(W_c^{\text{dec}}\,\boldsymbol{y}_{t-1} + U_c^{\text{dec}}\,\boldsymbol{h}_{t-1}^{\text{dec}} + V_c\,\boldsymbol{z} + \boldsymbol{b}_c^{\text{dec}}\Big) \tag{A.14}$$
$$\boldsymbol{c}_t^{\text{dec}} = \boldsymbol{f}_t^{\text{dec}} \odot \boldsymbol{c}_{t-1}^{\text{dec}} + \boldsymbol{i}_t^{\text{dec}} \odot \tilde{\boldsymbol{c}}_t^{\text{dec}} \tag{A.15}$$
$$\boldsymbol{h}_t^{\text{dec}} = \boldsymbol{o}_t^{\text{dec}} \odot \tanh\Big(\boldsymbol{c}_t^{\text{dec}}\Big) \tag{A.16}$$

Note that $\boldsymbol{z}$ is the same vector at every decode step.

A.4 Setup 3: Attention-based dynamic context vector

The encoder is unchanged (Eq. A.7). At each decode step t, the decoder computes a *step-specific* context vector $\boldsymbol{z}_t$ by attending over all n encoder hidden states. Using the **previous** decoder hidden state $\boldsymbol{h}_{t-1}^{\text{dec}}$ as the query:

$$e_{t,i} = \boldsymbol{h}_{t-1}^{\text{dec}} \cdot \boldsymbol{h}_i^{\text{enc}}, \qquad i = 1, \ldots, n \tag{A.17}$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{n} \exp(e_{t,j})} \tag{A.18}$$

$$\boldsymbol{z}_t = \sum_{i=1}^{n} \alpha_{t,i}\, \boldsymbol{h}_i^{\text{enc}} \tag{A.19}$$

Note that $e_{t,i}$ is a *dot product* between two vectors of the same dimension (`hidden_size`), and the resulting attention weight $\alpha_{t,i}$ is a scalar that measures how much attention the decoder pays to encoder position i when generating output token t. The decoder is initialised as in Setup 1 (Eq. A.8), and the cell equations have the same form as Setup 2 (Eqs. A.11–A.16) but with $\boldsymbol{z}_t$ in place of $\boldsymbol{z}$:

$$\boldsymbol{f}_t^{\text{dec}} = \sigma\Big(W_f^{\text{dec}}\, \boldsymbol{y}_{t-1} + U_f^{\text{dec}}\, \boldsymbol{h}_{t-1}^{\text{dec}} + V_f\, \boldsymbol{z}_t + \boldsymbol{b}_f^{\text{dec}}\Big) \tag{A.20}$$

$$\boldsymbol{i}_t^{\text{dec}} = \sigma\Big(W_i^{\text{dec}}\, \boldsymbol{y}_{t-1} + U_i^{\text{dec}}\, \boldsymbol{h}_{t-1}^{\text{dec}} + V_i\, \boldsymbol{z}_t + \boldsymbol{b}_i^{\text{dec}}\Big) \tag{A.21}$$

$$\boldsymbol{o}_t^{\text{dec}} = \sigma\Big(W_o^{\text{dec}}\, \boldsymbol{y}_{t-1} + U_o^{\text{dec}}\, \boldsymbol{h}_{t-1}^{\text{dec}} + V_o\, \boldsymbol{z}_t + \boldsymbol{b}_o^{\text{dec}}\Big) \tag{A.22}$$

$$\tilde{\boldsymbol{c}}_t^{\text{dec}} = \tanh\Big(W_c^{\text{dec}}\, \boldsymbol{y}_{t-1} + U_c^{\text{dec}}\, \boldsymbol{h}_{t-1}^{\text{dec}} + V_c\, \boldsymbol{z}_t + \boldsymbol{b}_c^{\text{dec}}\Big) \tag{A.23}$$

$$\boldsymbol{c}_t^{\text{dec}} = \boldsymbol{f}_t^{\text{dec}} \odot \boldsymbol{c}_{t-1}^{\text{dec}} + \boldsymbol{i}_t^{\text{dec}} \odot \tilde{\boldsymbol{c}}_t^{\text{dec}} \tag{A.24}$$

$$\boldsymbol{h}_t^{\text{dec}} = \boldsymbol{o}_t^{\text{dec}} \odot \tanh\Big(\boldsymbol{c}_t^{\text{dec}}\Big) \tag{A.25}$$

The key difference from Setup 2 is that $\boldsymbol{z}_t$ is recomputed from scratch at every decode step, so the decoder can focus on different encoder positions as it generates different output phonemes.